National University of Computer & Emerging Sciences

Karachi Campus — Spring 2026

Object Oriented Programming — CS1004

Assignment # 2 — Documentation Report

PakWheels-Style Car Marketplace System

Student ID	25K-0532
Subject	Object Oriented Programming — CS1004
Assignment	Assignment #2 — Advanced OOP Concepts
Due Date	18 April 2026, 11:59 PM
Language	C++17
Files	CarMarketplace2.h | IDisplayable.h | IPriceable.h | ISearchable.h | IReviewable.h | main.cpp

1. INHERITANCE

Overview

Inheritance allows a derived class to acquire attributes and behaviours from a base class, promoting code reuse and establishing an is-a relationship. In this assignment the existing A1 class hierarchy was extended to add two new inheritance chains: Vehicle -> Car / Bike, and a 5th User subtype. In total 12 inheritance relationships are implemented across the codebase.

Inheritance Relationships Table

#	Base Class	Derived Class	Relationship Type	Justification
1	Vehicle	Car	Public (Single)	Car is a Vehicle with car-specific attributes: doors, transmission, bodyType, sunroof. All Vehicle data (brand, price, mileage) is reused.
2	Vehicle	Bike	Public (Single)	Bike is a Vehicle with bike-specific data: bikeType, engineCC, hasGear. All Vehicle behaviour is inherited unchanged.
3	IDisplayable	Vehicle	Public (Interface)	Vehicle must define displayDetails(). Each derived type (Car, Bike) then overrides it with its own formatted output.
4	IPriceable	Vehicle	Public (Interface)	Vehicle must implement getPrice() and applyDiscount(), guaranteeing all vehicles are priceable through a common interface.
5	ISearchable	Vehicle	Public (Interface)	Vehicle must implement matchesSearch(keyword) so the marketplace can search any vehicle type uniformly.
6	IReviewable	Vehicle	Public (Interface)	Vehicle must implement addReview() and getRating() so ratings are supported on all vehicle types.
7	User	Seller	Public (Single)	Seller is a User with listing management, revenue tracking, and sales recording — a specialised user type.
8	User	Buyer	Public (Single)	Buyer is a User with a budget, favourite listings, and an inbox — a distinct user role sharing User's contact and address.
9	User	Admin	Public (Single)	Admin is a User with moderation powers (approve/reject listings) and a constant admin code assigned at creation.
10	IDisplayable	User	Public (Interface)	Seller, Buyer, and Admin each override displayDetails() to show role-specific information via a common pointer.
11	ISearchable	User	Public (Interface)	Users can be searched by username or userType, enabling polymorphic search across all user roles.
12	IPriceable	Listing	Public (Interface)	A Listing wraps a Vehicle and must expose getPrice() and applyDiscount(), delegating to its composed Vehicle.

Code Snippet — Car derived from Vehicle

```
class Car : public Vehicle {
private:
    int    numDoors;
    string transmission;
    string bodyType;       // Sedan, SUV, Hatchback, etc.
    bool   hasSunroof;

public:
    // Default constructor
    Car() : Vehicle(), numDoors(4), transmission("Automatic"),
            bodyType("Sedan"), hasSunroof(false) {
        vehicleType = "Car";
    }

    // Parameterized constructor (Inheritance #1: calls Vehicle constructor)
    Car(string br, string mo, int yr, double pr, int mi, string col,
        int doors, string trans, string body, bool sunroof = false)
        : Vehicle(br, mo, yr, pr, mi, col),
          numDoors(doors), transmission(trans),
          bodyType(body), hasSunroof(sunroof) {
        vehicleType = "Car";
    }
```

Code Snippet — Bike derived from Vehicle

```
class Bike : public Vehicle {
private:
    string bikeType;    // Sports, Cruiser, Commuter, Off-road
    int    engineCC;
    bool   hasGear;
    string brand125cc;  // Popular 125cc brands in Pakistan

public:
    // Default constructor
    Bike() : Vehicle(), bikeType("Commuter"), engineCC(125),
             hasGear(true), brand125cc("") {
        vehicleType = "Bike";
    }

    // Parameterized constructor (Inheritance #2: calls Vehicle constructor)
    Bike(string br, string mo, int yr, double pr, int mi, string col,
         string type, int cc, bool gear)
        : Vehicle(br, mo, yr, pr, mi, col),
          bikeType(type), engineCC(cc), hasGear(gear), brand125cc("") {
        vehicleType = "Bike";
    }
```

2. POLYMORPHISM

Overview

Polymorphism enables a single interface to represent different underlying types. Two forms are used:

- Function Overriding (Runtime Polymorphism) — derived classes override base virtual functions. A Vehicle* pointer calls the correct displayDetails() at runtime based on whether it points to a Car or Bike.
- Function Overloading (Compile-time Polymorphism) — the same function name is defined multiple times in the same class with different parameter lists.

Runtime Polymorphism — Function Overriding

Each of the four pure virtual functions from the abstract interfaces is overridden in the derived classes. The most visible example is displayDetails():

```
Vehicle* vPtr;


vPtr = &car1;   vPtr->displayDetails();   // Calls Car::displayDetails()
vPtr = &car2;   vPtr->displayDetails();   // Calls Car::displayDetails()
vPtr = &bike1;  vPtr->displayDetails();   // Calls Bike::displayDetails()
vPtr = &bike2;  vPtr->displayDetails();   // Calls Bike::displayDetails()
```

The same pattern applies via IDisplayable*, demonstrating that any User subtype also participates in runtime polymorphism:

```
IDisplayable* dispPtr;
dispPtr = &seller1;  dispPtr->displayDetails();
dispPtr = &buyer1;   dispPtr->displayDetails();
dispPtr = &admin1;   dispPtr->displayDetails();
```

Compile-time Polymorphism — Function Overloading

Three overloaded versions of displayDetails() are added across the derived classes:

Car::displayDetails()	void displayDetails() const — standard output with doors, transmission, sunroof
Car::displayDetails(bool)	void displayDetails(bool showReviews) — also prints all reviews when true
Bike::displayDetails()	void displayDetails() const — standard output with bikeType, cc, gear
Bike::displayDetails(bool)	void displayDetails(bool compact) — one-liner compact format when true
Seller::displayDetails()	void displayDetails() const — profile + listing count + revenue
Seller::displayDetails(bool)	void displayDetails(bool showListings) — also prints all listings when true

Buyer::displayDetails()	void displayDetails() const — profile + budget + favourites count
Buyer::displayDetails(bool)	void displayDetails(bool showFavs) — also prints favourite listings when true

3. ABSTRACTION

Overview

Abstraction hides internal implementation and exposes only essential behaviour through interfaces. Four pure abstract classes (interfaces) are defined, each in its own header file. Any class that inherits from them must implement all pure virtual functions or it cannot be instantiated.

Abstract Class 1 — IDisplayable.h

```cpp
#pragma once
#include <iostream>
using namespace std;

// ============================================================
// ABSTRACT CLASS 1: IDisplayable
// Purpose: Forces every derived class to implement its own
//          displayDetails() function. This is pure Abstraction —
//          we define WHAT must be done, not HOW.
// Used by: Vehicle (Car, Bike), User (Buyer, Seller, Admin)
// ============================================================
class IDisplayable {
public:
    virtual void displayDetails() const = 0;   // Pure virtual function
    virtual ~IDisplayable() {}
};
```

Reasoning: Every entity in the system (vehicles, users) must be printable. By forcing displayDetails() to be overridden, the abstract class guarantees uniform display behaviour while letting each class format output as it sees fit. The Marketplace can iterate an array of IDisplayable* pointers without knowing the concrete type.

Abstract Class 2 — IPriceable.h

```cpp
#pragma once

// ============================================================
// ABSTRACT CLASS 2: IPriceable
// Purpose: Any entity with a price must implement getPrice()
//          and applyDiscount(). Hides HOW pricing works —
//          only the interface is visible to the outside world.
// Used by: Vehicle (Car, Bike), Listing
// ============================================================
class IPriceable {
public:
    virtual double getPrice()              const = 0;  // Pure virtual
    virtual void   applyDiscount(double pct)     = 0;  // Pure virtual
    virtual ~IPriceable() {}
};
```

Reasoning: Both Vehicle and Listing expose a price and support discounts. IPriceable lets any discount-management system operate on either type without knowing which it holds. Car overrides applyDiscount() with luxury-tax logic; Bike uses the inherited Vehicle implementation.

Abstract Class 3 — ISearchable.h

```cpp
#pragma once
#include <string>
using namespace std;

// ============================================================
// ABSTRACT CLASS 3: ISearchable
// Purpose: Enables polymorphic searching across all entity types.
//          Each class defines its own matching logic internally.
//          The marketplace only needs to call matchesSearch().
// Used by: Car, Bike, Seller, Buyer
// ============================================================
class ISearchable {
public:
    virtual bool matchesSearch(const string& keyword) const = 0;  // Pure virtual
    virtual ~ISearchable() {}
};
```

Reasoning: The marketplace needs to search across heterogeneous types — Cars, Bikes, and Users. ISearchable provides a single interface; each class implements its own keyword matching (Car also searches bodyType and transmission; Bike also searches engineCC).

Abstract Class 4 — IReviewable.h

```cpp
#pragma once
#include <string>
using namespace std;

// ============================================================
// ABSTRACT CLASS 4: IReviewable
// Purpose: Any entity that can receive user ratings/reviews
//          must define addReview() and getRating().
//          Hides internal storage of reviews from outside code.
// Used by: Vehicle (Car, Bike), Seller, Buyer
// ============================================================
class IReviewable {
public:
    virtual void   addReview(const string& comment, int rating) = 0;  // Pure virtual
    virtual double getRating()                                  const = 0;  // Pure virtual
    virtual ~IReviewable() {}
};
```

Reasoning: Both vehicles and sellers can receive reviews. IReviewable hides whether reviews are stored in arrays, vectors, or a database. A rating engine uses IReviewable* pointers and never needs to know the concrete type underneath.

Interface Usage via Pointers (Abstraction in Action)

```
IPriceable* pricePtr;
pricePtr = &car1;     pricePtr->applyDiscount(5.0);
pricePtr = &bike1;    pricePtr->applyDiscount(3.0);
pricePtr = &l2;       pricePtr->applyDiscount(2.0);
```

4. OPERATOR OVERLOADING

Overview

Operator overloading allows C++ operators to work with user-defined types in a natural, readable way. Below are all 17 overloaded operators across the system. Some are implemented as member functions (can access this directly) and some as global friend functions (need access to private data of multiple objects or must match standard stream conventions).

Member vs Global (Friend) Operator Rationale

- Member operators: Used when the left-hand operand is always the current class (e.g., a == b, a < b, a + value). The operator naturally accesses 'this' object's private data.
- Global friend operators: Used for operator<< and operator>> since ostream/istream is the left operand, not the class itself. Making them friend grants access to private fields without exposing them publicly.

Complete Operator Overloading Table

#	Operator	Class	Type	Signature / Usage	Reasoning
1	`==`	`Address`	`Member`	`addr1 == addr2`	Checks same city+province. Used to match buyer and seller locations.
2	`!=`	`Address`	`Member`	`addr1 != addr2`	Inverse of ==; used to filter cross-city searches.
3	`<<`	`Address`	`Global Friend`	`cout << address`	Stream output. Left operand is ostream, so must be global; friend for private field access.
4	`==`	`ContactInfo`	`Member`	`c1 == c2`	Matches by phone number. Detects duplicate registrations before adding to system.
5	`<<`	`ContactInfo`	`Global Friend`	`cout << contact`	Clean stream output. Must be global because ostream is the left operand.
6	`<<`	`Message`	`Global Friend`	`cout << msg`	Prints message in one line with timestamp. Global friend accesses all private fields.
7	`==`	`Vehicle`	`Member`	`v1 == v2`	Two vehicles are equal if brand+model+year match. Prevents duplicate listings.
8	`<`	`Vehicle`	`Member`	`v1 < v2`	Price comparison. Used to sort search results from cheapest to most expensive.
9	`>`	`Vehicle`	`Member`	`v1 > v2`	Inverse price comparison. Used to sort results from most to least expensive.
10	`<<`	`Vehicle`	`Global Friend`	`cout << vehicle`	Single-line vehicle summary. Global friend accesses price, mileage, color directly.
11	`==`	`Car`	`Member`	`car1 == car2`	Extends Vehicle == by also comparing bodyType. Two Cars are only equal if all four fields match.
12	`+`	`Bike`	`Member`	`bundle = bike1 + bike2`	Returns combined price of two bikes for bundle deal calculation.

13	==	Listing	Member	l1 == l2	Compares by listingId. Prevents duplicate insertions into the marketplace.
14	<<	Listing	Global Friend	cout << listing	Compact one-line listing display. Friend accesses private listingId and sellerUsername.
15	==	User	Member	u1 == u2	Compares by username. Prevents two accounts with the same username.
16	<<	User	Global Friend	cout << user	Prints userType and username. Friend accesses private username and isVerified.
17	+	Buyer	Member	buyer2 = buyer1 + 500000	Returns a new Buyer with increased budget. Simulates adding funds to an account.

Code Snippet — Selected Operator Implementations

```
bool operator==(const Car& other) const {
    return brand == other.brand && model == other.model &&
           year  == other.year  && bodyType == other.bodyType;
}
```

```
friend ostream& operator<<(ostream& os, const Vehicle& v) {
    os << v.year << " " << v.brand << " " << v.model
       << " | PKR " << v.price << " | " << v.mileage << " km | " << v.color;
    return os;
}
```

```
double operator+(const Bike& other) const {
    return price + other.price;
}
```

5. FRIEND FUNCTIONS & FRIEND CLASSES

Overview

Friend functions and classes are granted direct access to the private members of a class. This is appropriate when an operation logically involves multiple objects of the same or different classes, and requiring a public getter for every field would break encapsulation for all callers rather than just the trusted one.

Rule of thumb applied: if a function needs private data of TWO objects simultaneously, or if the operation is sensitive enough that we do not want a public setter but need controlled access, it is made a friend.

Friend Functions & Classes Table

#	Name	Type	Accesses Private Of	Justification
1	comparePrices()	Friend Function	Vehicle (price)	Needs the private 'price' field of BOTH Vehicle objects simultaneously. A member function can only access 'this->price'. Making it a friend is the only way to read both prices without adding a public getter used only for this purpose.
2	printVehicleSummary()	Friend Function	Vehicle (brand, model, price, mileage, reviewCount)	Needs formatted read access to five private fields at once for a diagnostic report. Avoids five separate getter calls; friend access keeps the code clean and the fields private to all other non-trusted callers.
3	compareEngineCC()	Friend Function	Bike (engineCC)	Needs the private engineCC of TWO Bike objects. As a non-member it has no access to engineCC. Making it a friend is the minimal change that grants access to exactly these two private fields and nothing more.
4	transferListing()	Friend Function	Listing (sellerUsername, status)	Performs a sensitive ownership transfer. We deliberately do NOT add a public setSellerUsername() because that would allow any code to reassign ownership. Friend access restricts this operation to one trusted function only.
5	Admin (class)	Friend Class	Car (numDoors, hasSunroof, bodyType, transmission)	Admin is the platform moderator; it needs to validate car specifications directly during the listing review process. Rather than exposing numDoors and hasSunroof publicly (available to all buyers), Admin is granted friend access so only it can read these fields.

6. ADDITIONAL REQUIREMENTS & INTEGRATION

Integration with Assignment 1

The Assignment 2 code fully preserves all Assignment 1 functionality:

- All original classes (Address, ContactInfo, Message, Vehicle, CarSpecifications, Listing, User, Seller, Buyer, Admin, SearchFilter, Marketplace) are retained with identical public interfaces.
- The original main.cpp test scenarios (seller registration, listings, admin moderation, buyer search, favourites, messaging, profile display, copy constructors) all run correctly.
- New classes (Car, Bike) extend Vehicle and are used in place of the plain Vehicle object where vehicle-type-specific behaviour is needed.
- Four abstract interface headers (IDisplayable.h, IPriceable.h, ISearchable.h, IReviewable.h) are added as separate files; CarMarketplace2.h includes them.

File Structure

IDisplayable.h	Abstract class — pure virtual displayDetails()
IPriceable.h	Abstract class — pure virtual getPrice() and applyDiscount()
ISearchable.h	Abstract class — pure virtual matchesSearch()
IReviewable.h	Abstract class — pure virtual addReview() and getRating()
CarMarketplace2.h	All concrete classes: Address, ContactInfo, Message, Vehicle, Car, Bike, CarSpecifications, Listing, User, Seller, Buyer, Admin, SearchFilter, Marketplace
main.cpp	Demonstrates all A1 + A2 features with clear section headers

The code compiles with zero errors and zero warnings under C++17. All 6 requirement sections are demonstrated with labelled output in main.cpp.